SPPU_TE_BE_COMP 👈 Click to join telegram

---

1. Hashing definition.

Hashing

Definition

Hashing is a technique used to **map a key value to a fixed-size index (address)** in a **hash table** using a **hash function**, enabling **fast insertion, deletion, and searching**.

Formally:

$$\text{Address} = h(\text{key})$$

where `h` is the hash function.

Key Components of Hashing

1. Hash Table

- An array-like data structure used to store data
- Index is computed using a hash function

2. Hash Function

- A function that converts a key into an index
- Should be **fast**, **deterministic**, and **uniform**

Example: `h(k) =`

```
k mod 10
```

---

3. Key

- The value to be stored or searched (roll number, ID, name, etc.)

Example

Given:

```
Table size = 10
Key = 27
Hash function h(k) = k mod 10

h(27) = 27 mod 10 = 7
```

So, key `27` is stored at **index 7**.

Why Hashing is Used

- Provides **O(1)** average time complexity
- Faster than searching in arrays, linked lists, or trees
- Efficient for large datasets

Applications of Hashing

- Symbol tables
- Database indexing
- Password storage
- Caches (browser, CPU cache)
- Compilers

---

2. Good hash function properties

Properties of a Good Hash Function

A **good hash function** is essential for efficient hashing. It should distribute keys **uniformly** in the hash table and minimize **collisions**.

SPPU_TE_BE_COMP 👈 Click to join telegram

Properties

1. Uniform Distribution

- Hash values should be **evenly spread** over the hash table
- Reduces collisions

---

2. Deterministic

- Same key must always produce the **same hash value**

$$h(k) \rightarrow \text{same index every time}$$

---

3. Fast Computation

- Hash function should be **quick to compute**
- Important for real-time performance

---

4. Minimize Collisions

- Different keys should ideally map to **different indices**

---

5. Uses All Key Information

- Hash value should depend on **all parts of the key**
- Avoids clustering

---

6. Simple to Implement

- Easy to understand and implement
- Less computational overhead

---

7. Insensitive to Input Patterns

- Similar keys should not map to similar hash values

Example of a Good Hash Function

`h(k) = (k × 31) mod table_size`

Bad Hash Function Example

`h(k) = k mod 10  (for similar keys like 20, 30, 40)`

3. Modulo hashing

Modulo Hashing

Definition

Modulo hashing is a hashing technique in which the **hash value (index)** is obtained by taking the **remainder** when a key is divided by the **table size**.

$$h(k) = k \bmod m$$

where:

- `k` = key
- `m` = size of hash table

How Modulo Hashing Works

Example

- Hash table size `m = 10`
- Key `k = 47`

$$h(47) = 47 \bmod 10 = 7$$

➡ Store key `47` at **index 7**.

Another Example

Keys: `23, 44, 12, 87`
Table size: `10`

Key	Hash value
23	3
44	4
12	2
87	7

Advantages

- Very **simple and fast**
- Easy to implement
- Requires minimal computation

Disadvantages

- **High collision probability** if `m` is poorly chosen
- Not suitable when keys follow patterns
- Bad choice of `m` (like powers of 2) leads to clustering

---

4. Mid-square method

Mid-Square Method (Hashing)

Definition

The **mid-square method** is a hashing technique in which the **key is squared**, and then the **middle digits of the result** are taken as the **hash value**.
This method helps in achieving **better distribution** of hash values.

Steps of Mid-Square Method

1. Take the **key**
2. **Square** the key
3. Extract the **middle digits** of the squared value
4. Use those digits as the **hash index**

Example

Given:

- Key `k = 44`
- Hash table size = 100 (2 digits)

Step 1: Square the key

$$44^2 = 1936$$

Step 2: Extract middle digits

- Middle two digits = **93**

☑ Hash value: `h(44)`

`= 93`

Another Example

Key = `23`

232=52923^2 = 529232=529

Middle digit = **2**

➡ `h(23) = 2`

Advantages

- Uses **all bits** of the key
- Less affected by patterns in keys
- Produces better uniform distribution

Disadvantages

- Extra computation (squaring)
- Choosing correct middle digits can be tricky
- Not ideal for very large keys

Applications

- Educational hash tables
- When keys show patterns (like sequential numbers)

---

5. Folding method.

Folding Method (Hashing)

Definition

The **folding method** is a hashing technique in which the **key is divided into equal-sized parts**, and these parts are **added together** (or combined) to produce the **hash value**.

Types of Folding Method

1. Shift Folding

- Divide the key into parts
- Add all parts directly

2. Boundary Folding

- Reverse alternate parts before adding

Steps (General)

1. Divide the key into **equal-sized blocks**
2. Add the blocks together
3. If required, apply modulo with table size
4. Result is the **hash index**

Example 1: Shift Folding

Given:

- Key = `12345678`
- Block size = 2 digits **Blocks:**

`12 | 34 | 56 | 78` **Sum:**

`12 + 34 + 56 + 78 = 180`

If table size = 100:

`h(k) = 180 mod 100 = 80`

Example 2: Boundary Folding

Blocks:

`12 | 34 | 56 | 78`

Reverse alternate blocks:

`12 | 43 | 56 | 87` Sum:

```
12 + 43 + 56 + 87 = 198
```

Advantages

- Simple and easy to implement
- Uses all digits of the key
- Reduces effect of similar keys

Disadvantages

- Collisions still possible
- Depends on block size choice
- Not suitable for very small keys

Applications

- Telephone numbers
- Account numbers
- Identification numbers

---

6. Collision definition.

---

collision (in Hashing):

A **collision** occurs when **two or more different keys produce the same hash value (or hash index)** when processed by a hash function.
Since each hash table index can normally store only one key, a collision creates a conflict that must be resolved using a collision-handling technique.

Definition (Exam-oriented):

Collision is the situation in hashing where multiple distinct keys are mapped to the same location in the hash table.

Example:
If a hash function is

$$h(k) = k \bmod 10$$

Then:

- $h(25) = 5$
- $h(15) = 5$

Here, keys **25** and **15** collide because they map to the same index **5**.

Why collisions occur:

- Limited size of hash table
- Poor hash function
- Large number of keys compared to table size

Common collision resolution methods:

1. Chaining
2. Linear probing
3. Quadratic probing
4. Double hashing

---

7. Chaining method

Chaining Method (Collision Resolution Technique)

Definition:
The **chaining method** is a collision resolution technique in hashing where **each index of the hash table contains a linked list (chain)**.
When two or more keys hash to the same index, they are **stored in the same location as a chain**.

SPPU_TE_BE_COMP 👈 Click to join telegram

How Chaining Works

1. Apply the hash function to the key.
2. If the computed index is empty, insert the key.
3. If the index already contains a key, **add the new key to the linked list** at that index.

Example

Hash function:

$$h(k) = k \bmod 5$$

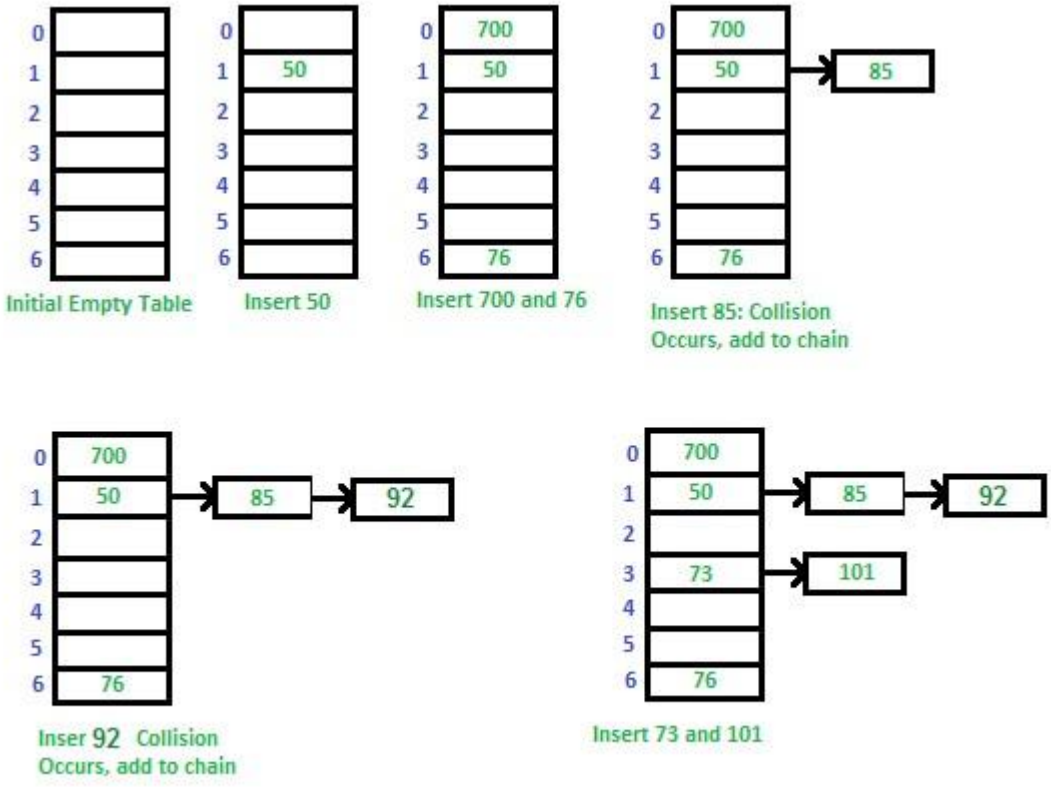
Keys: **12, 22, 32, 7**

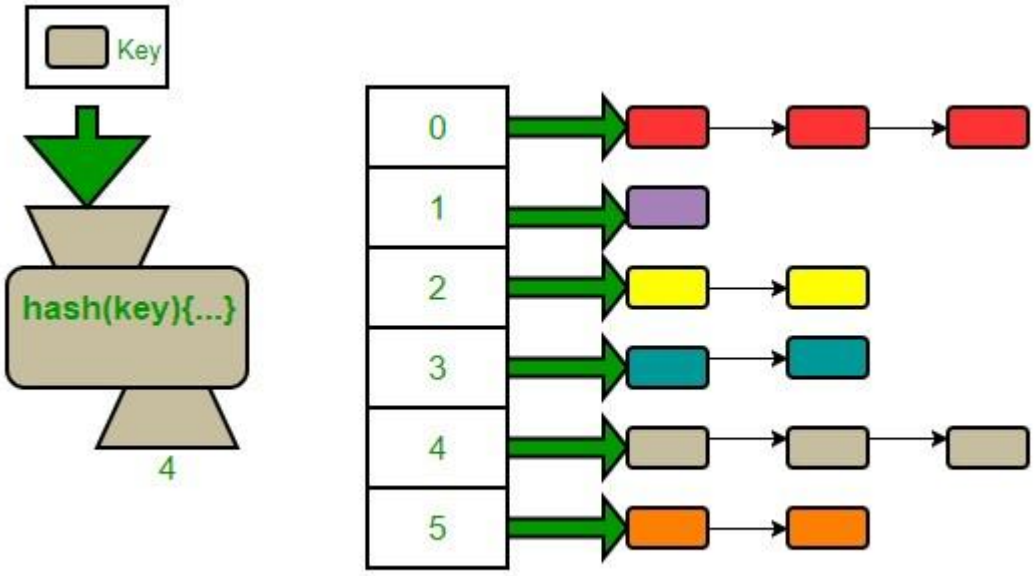
Key	Hash Value
12	2
22	2
32	2
7	2

Hash Table using Chaining:

Index 2 → 12 → 22 → 32 → 7 → NULL

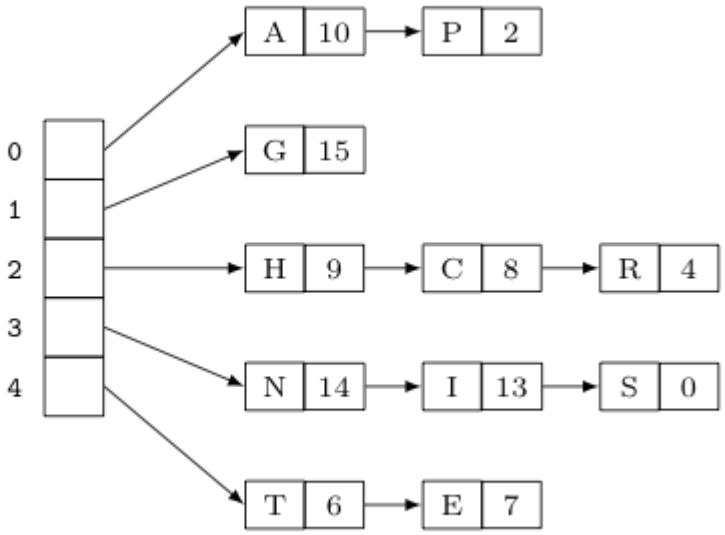
Diagram


0
1
2
3
4
5
6
Initial Empty Table
50
Insert 50
700
76
Insert 700 and 76
85
Insert 85: Collision
Occurs, add to chain
92
Inser 92 Collision
Occurs, add to chain
73
101
Insert 73 and 101


Key
hash(key){...}
4
0
1
2
3
4
5

SPPU_TE_BE_COMP Click to join telegram

key	slot	value
S	3	0
E	4	1
P	0	2
A	0	3
R	2	4
A	0	5
T	4	6
E	4	7
C	2	8
H	2	9
A	0	10
I	3	11
N	3	12
I	3	13
N	3	14
G	1	15



Advantages

- Simple and easy to implement
- No limitation on number of collisions
- Hash table never becomes completely full

Disadvantages

- Extra memory required for linked lists
- Performance degrades if many keys map to the same index
- Worst-case time complexity is **O(n)**

Time Complexity

- **Average case:** O(1)
- **Worst case:** O(n)

---

8. Open addressing

Open Addressing (Collision Resolution Technique)

Definition:
Open addressing is a collision resolution technique in hashing where **all keys are stored directly inside the hash table itself**.
When a collision occurs, the algorithm **searches for another empty slot** using a predefined probing sequence.

How Open Addressing Works

1. Compute the hash index using the hash function.
2. If the slot is empty → insert the key.
3. If a collision occurs → apply a **probing method** to find the next empty slot.
4. Continue probing until an empty position is found.

Types of Open Addressing

Types of Open Addressing

1. **Linear Probing**

$$h_i(k) = (h(k) + i) \bmod m$$

   - Checks the next slot sequentially.

2. **Quadratic Probing**

$$h_i(k) = (h(k) + i^2) \bmod m$$

   - Reduces primary clustering.

3. **Double Hashing**

$$h_i(k) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

   - Best distribution, least clustering.

Example (Linear Probing)

Hash function:

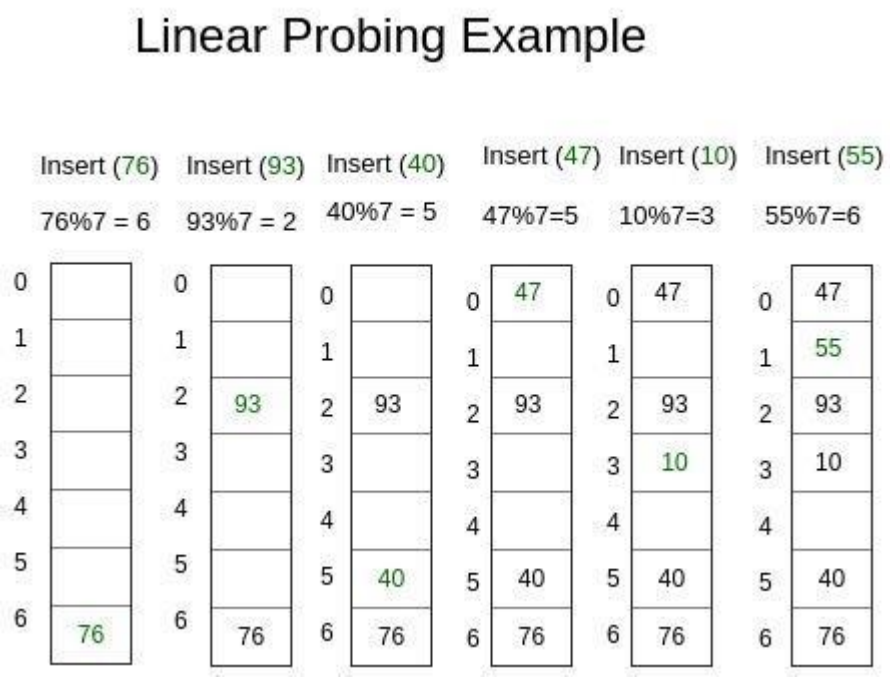
SPPU_TE_BE_COMP 👈 Click to join telegram

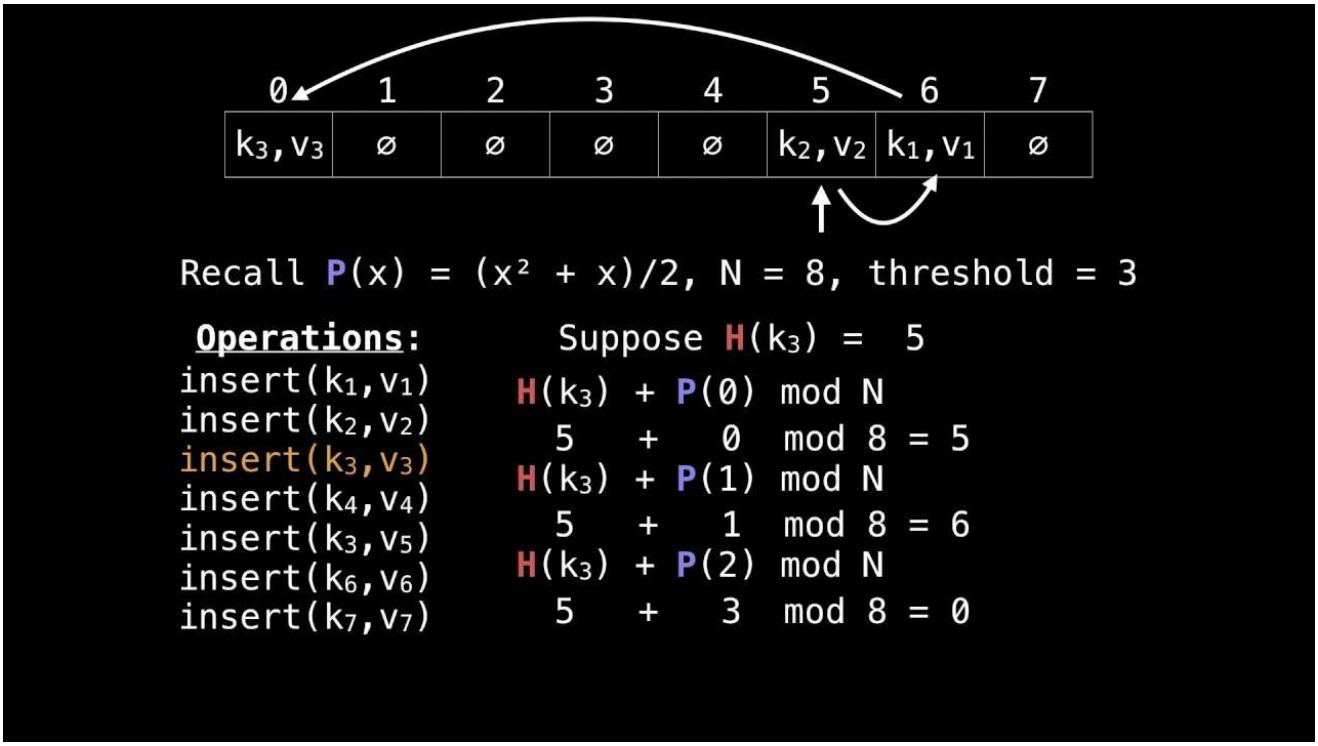
$$h(k) = k \bmod 7$$

Insert keys: **10, 17, 24**

Key	Initial Index	Final Position
10	3	3
17	3	4
24	3	5

Diagram





Advantages

- No extra memory needed (no linked lists)
- Better cache performance
- Simple structure

Disadvantages

- Hash table size must be larger
- Deletion is complex
- Performance decreases as table fills up
- Clustering problem

Time Complexity

- **Average case:** O(1)
- **Worst case:** O(n)

SPPU_TE_BE_COMP 👈 Click to join telegram

9. Linear probing.

Linear Probing (Collision Resolution Technique)

Definition:

Linear probing is a type of **open addressing** method used in hashing.
When a collision occurs, the algorithm **checks the next consecutive slots one by one** until an empty location is found.

Hash Function Formula

$$h_i(k) = (h(k) + i) \bmod m$$

Where:

- $h(k)$ = original hash value
- $i = 0, 1, 2, \ldots$ (probe sequence)
- m = size of hash table

Example

Hash table size = **7**

Hash function:

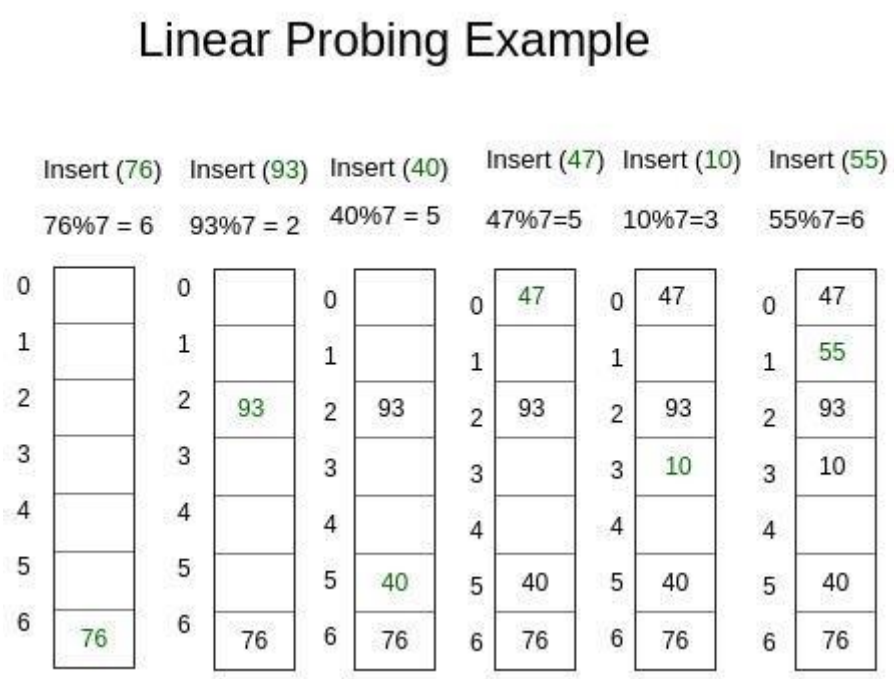
$$h(k) = k \bmod 7$$

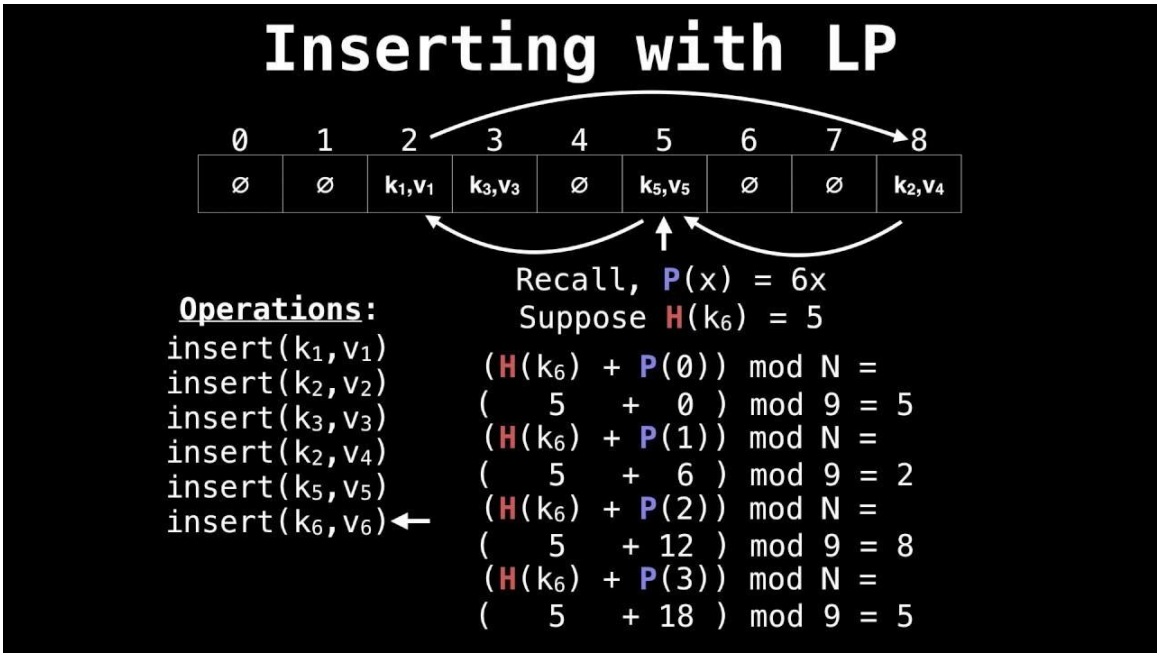
Insert keys: **10, 17, 24**

Key	Hash Value	Probing Result
10	3	Insert at index 3
17	3	Collision → index 4
24	3	Collision → index 5

SPPU_TE_BE_COMP 👈 Click to join telegram

Diagram





Advantages

- Easy to implement
- No extra memory required
- Good cache performance

Disadvantages

- **Primary clustering** occurs
- Performance degrades as table fills
- Deletion is complicated

Time Complexity

- **Average case:** O(1)
- **Worst case:** O(n)

---

10. Quadratic probing.

Definition:

Quadratic probing is a type of **open addressing** method used in hashing.

When a collision occurs, the algorithm searches for an empty slot by adding **successive square values** to the original hash index.

Hash Function Formula

$$h_i(k) = (h(k) + i^2) \bmod m$$

Where:

- $h(k)$ = original hash value
- $i = 0, 1, 2, 3, \ldots$ (probe number)
- m = size of hash table

SPPU_TE_BE_COMP 👈 Click to join telegram

How Quadratic Probing Works

1. Compute the initial hash index.
2. If the slot is empty → insert the key.
3. If collision occurs → check positions at distances $1^2, 2^2, 3^2, \ldots$.
4. Continue until an empty slot is found.

Example

Hash table size = **7**

Hash function:

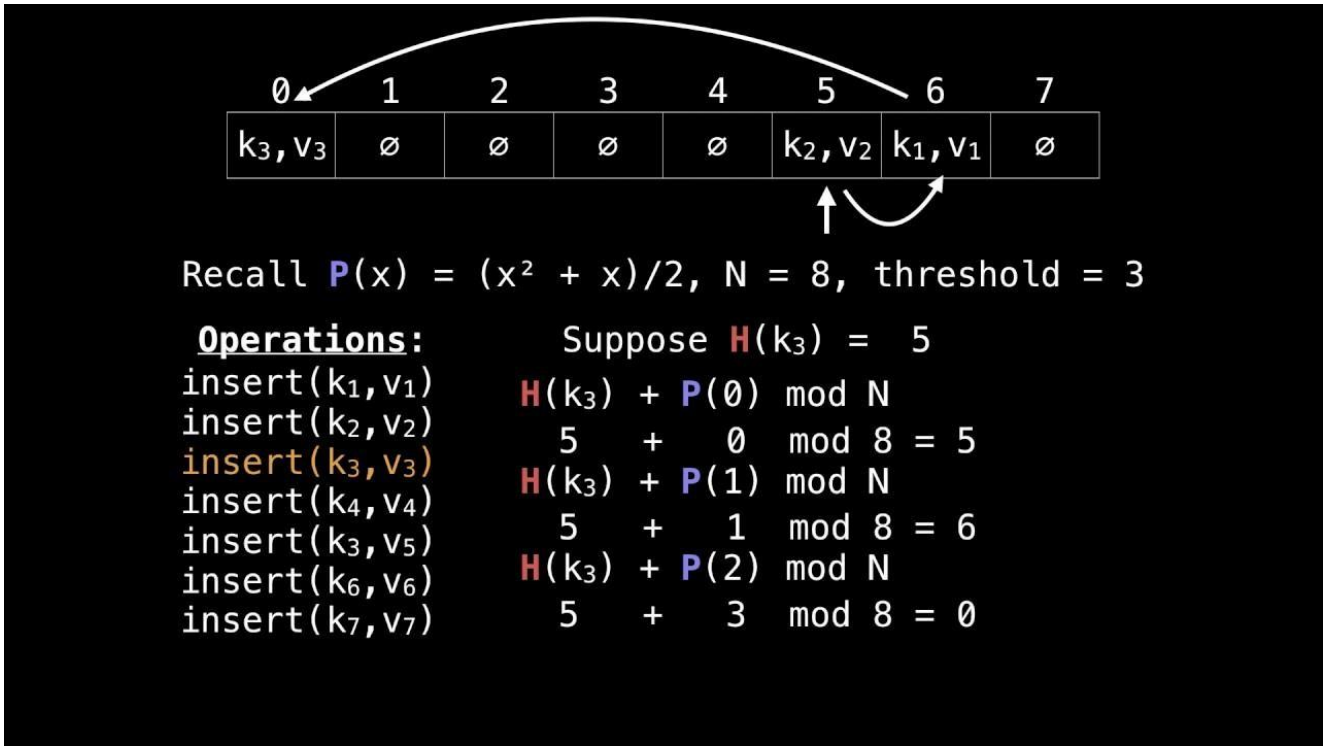
$$h(k) = k \bmod 7$$

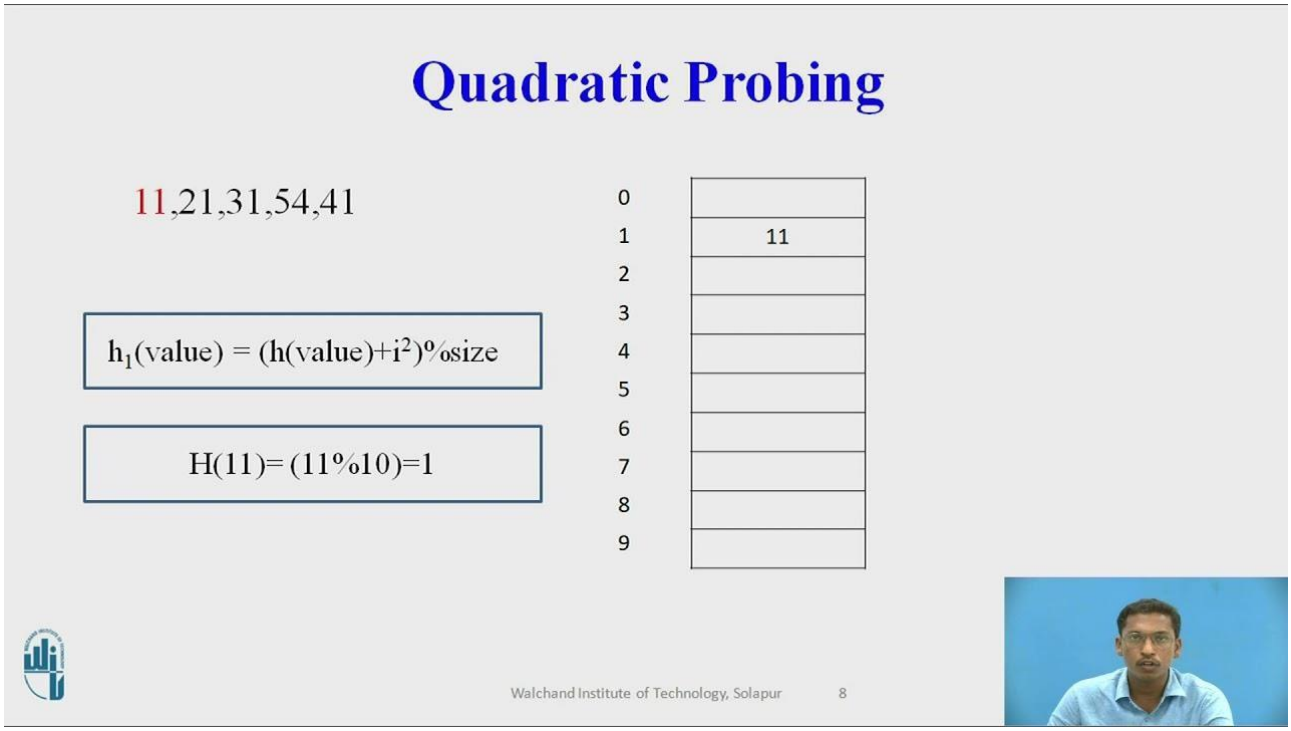
Insert keys: **10, 17, 24**

Key	Probing Sequence	Final Index
10	3	3
17	3 → 4	4
24	3 → 4 → 0	0

(Indices checked: $3 + 1^2 = 4$, $3 + 2^2 = 7 \equiv 0$)

Diagram





Advantages

- Reduces **primary clustering** compared to linear probing
- Better key distribution
- No extra memory required

Disadvantages

- **Secondary clustering** may still occur
- Table may not be fully utilized
- Choice of table size is important (preferably prime)

Time Complexity

- **Average case:** O(1)
- **Worst case:** O(n)

---

11. Double hashing.

Double Hashing (Collision Resolution Technique)

Definition:
Double hashing is an **open addressing** collision resolution technique in which **two different hash functions** are used.
When a collision occurs at the index produced by the first hash function, the **second hash function decides the step size** for probing.

`h1(k) = k mod m h2(k) = 1 + (k mod (m - 1)) hi(k) = (h1(k) + i * h2(k)) mod m` Where:

- `k` = key
- `m` = size of hash table (preferably prime)
- `i` = probe number (0, 1, 2, …)

How Double Hashing Works

1. Compute index using `h1(k)`.
2. If the slot is empty → insert the key.
3. If collision occurs → compute step size using `h2(k)`.
4. Probe new positions using the combined formula until an empty slot is found.

Example

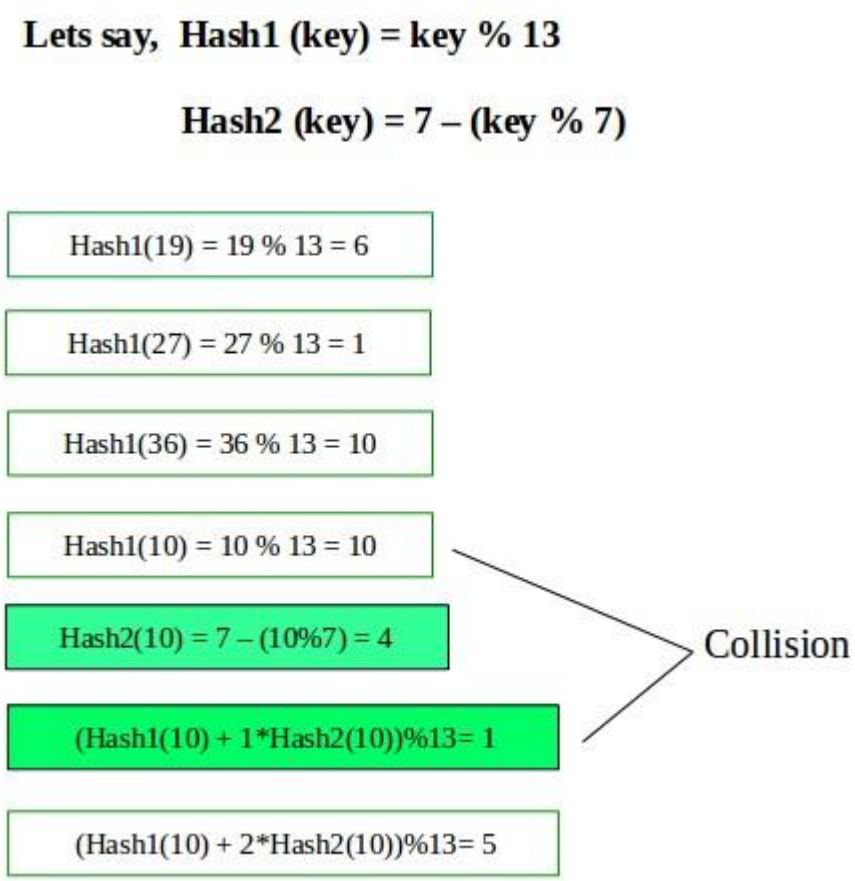
Hash table size: **7**

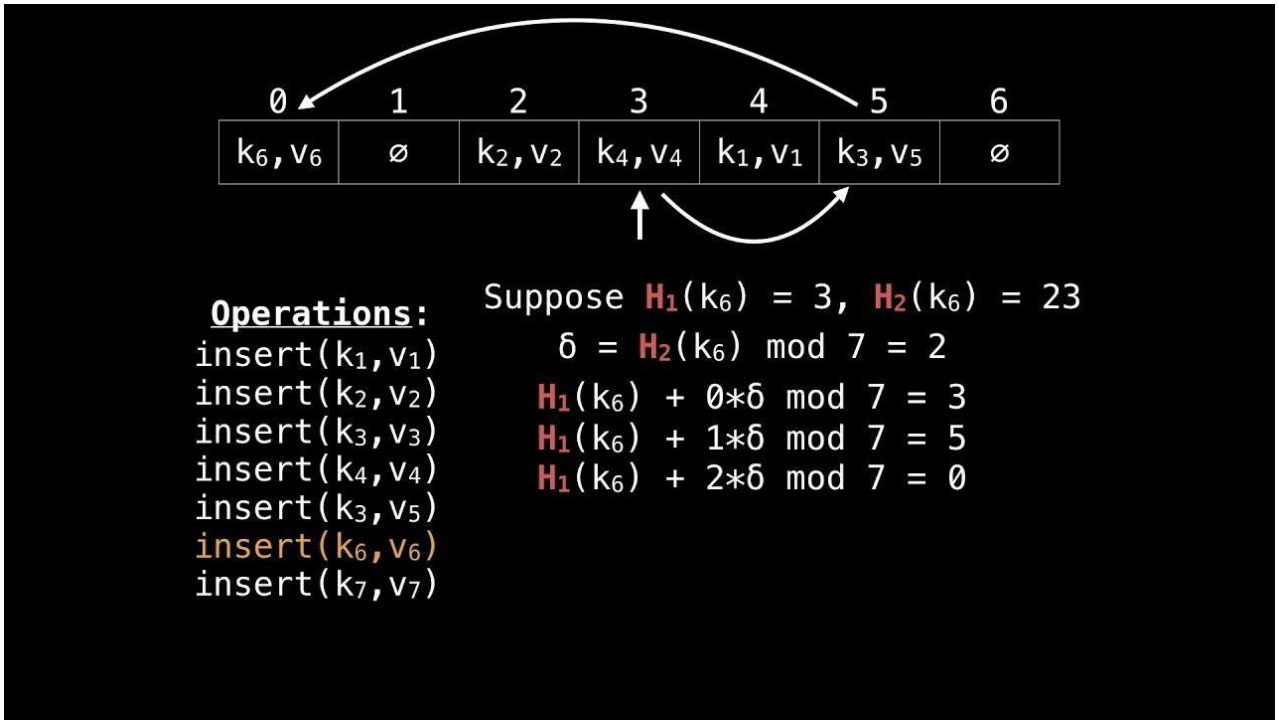
```
h1(k) = k mod 7 h2(k)
= 1 + (k mod 6)
```

Insert keys: **10, 17, 24**

Key	h1(k)	h2(k)	Probing Sequence	Final Index
10	3	5	3	3
17	3	6	3 → 2	2
24	3	1	3 → 4	4

Diagram


Lets say, Hash1 (key) = key % 13
Hash2 (key) = 7 – (key % 7)
Hash1(19) = 19 % 13 = 6
Hash1(27) = 27 % 13 = 1
Hash1(36) = 36 % 13 = 10
Hash1(10) = 10 % 13 = 10
Hash2(10) = 7 – (10%7) = 4
(Hash1(10) + 1*Hash2(10))%13= 1
(Hash1(10) + 2*Hash2(10))%13= 5
Collision


0 1 2 3 4 5 6
k6,v6 ∅ k2,v2 k4,v4 k1,v1 k3,v5 ∅
Operations:
insert(k1,v1)
insert(k2,v2)
insert(k3,v3)
insert(k4,v4)
insert(k3,v5)
insert(k6,v6)
insert(k7,v7)
Suppose H1(k6) = 3, H2(k6) = 23
δ = H2(k6) mod 7 = 2
H1(k6) + 0*δ mod 7 = 3
H1(k6) + 1*δ mod 7 = 5
H1(k6) + 2*δ mod 7 = 0

Advantages

- Minimizes both **primary and secondary clustering**
- Better distribution of keys
- Efficient even at higher load factors

Disadvantages

- More complex than linear and quadratic probing
- Requires two good hash functions
- Slightly more computation

Time Complexity

- **Average case:** O(1)
- **Worst case:** O(n)

SPPU_TE_BE_COMP 👈 Click to join telegram

12. Chaining vs open addressing

Basis	Chaining	Open Addressing
Basic idea	Stores collided keys in a **linked list** at the same index	Stores all keys **inside the hash table itself**
Extra memory	Required (linked lists)	Not required
Collision handling	Multiple keys allowed at same index	Finds another empty slot using probing
Load factor (α)	Can be **greater than 1**	Must be **less than 1**
Deletion	Easy	Difficult (needs special markers)
Clustering	No clustering	Suffers from clustering
Table overflow	No immediate overflow	Table can become full
Performance	Degrades with long chains	Degrades as table fills
Implementation	Simple	More complex

Examples	Hash table with linked lists	Linear, quadratic, double hashing

---

13. Naïve string matching

Naïve String Matching Algorithm

Definition:
The **naïve string matching algorithm** is a simple pattern searching technique that **checks the pattern at every possible position** in the given text and compares characters **one by one**.

Basic Idea

- Given a **text (T)** of length `n`
- Given a **pattern (P)** of length `m`
- The pattern is matched with the text starting from index `0` to `n - m`
- If all characters match → pattern found
- Else → shift pattern by **one position** and repeat

Algorithm (Word-2015 Friendly)

```
for i = 0 to n - m
j = 0
    while j < m and T[i + j] == P[j]
j = j + 1     if j == m
        print "Pattern found at position", i
```

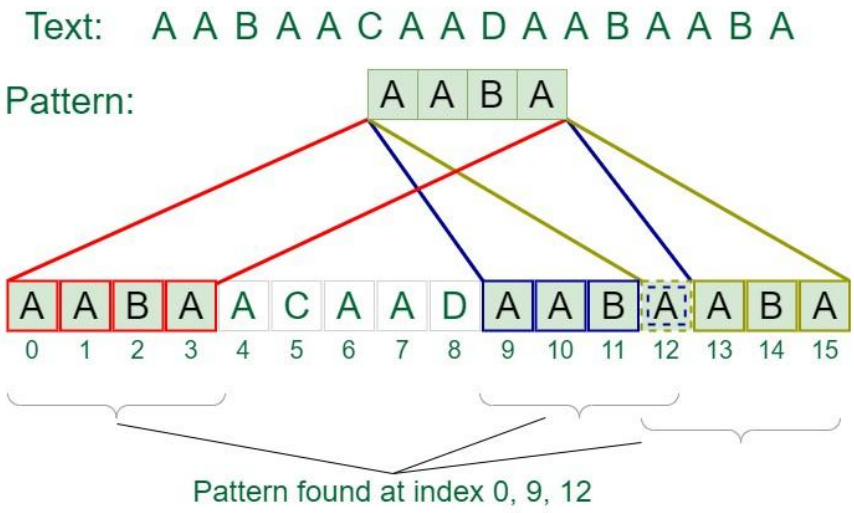
Example

```
Text (T)     = A B A B A B C
Pattern  (P)=   A   B   A   B
```

Matching steps:

- Compare P with T starting at index 0 → match found
- Pattern occurs at **position 0**

Diagram



Advantages

- Very easy to understand and implement
- No preprocessing required

Disadvantages

- Inefficient for large texts
- Repeated comparisons increase time

Time Complexity

- **Best case:** O(n)
- **Worst case:** O(n × m)

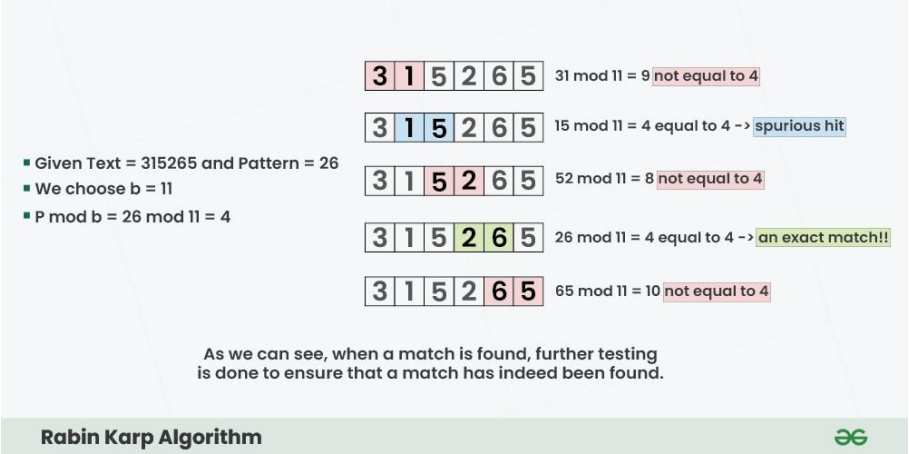
Where:

- n = length of text
- m = length of pattern

---

14. Rabin-Karp algorithm.

Rabin–Karp Algorithm



Definition

Rabin–Karp algorithm is a **string matching algorithm** that uses a **hashing technique** to find a **pattern** in a **text**.
It compares **hash values** of the pattern and text substrings instead of comparing characters one by one.

Idea of the Algorithm

- Compute the **hash value** of the **pattern**
- Compute the hash value of the **first window** of text
- Slide the window over the text (**rolling hash**)
- If hash values match → **compare characters**

- If characters match → **pattern found**

Notations

- `T` = Text of length `n`
- `P` = Pattern of length `m`
- `d` = Number of characters in alphabet (usually 10 or 256)
- `q` = A large **prime number** (to avoid collisions)

Hash Function Formula

$$h = (d^{m-1}) \bmod q$$

Pattern Hash Formula

$$p = (d \times p + P[i]) \bmod q$$

Text Window Hash Formula

$$t = (d \times t + T[i]) \bmod q$$

Rolling Hash Formula (Important)

$$t_{next} = (d \times (t - T[i] \times h) + T[i+m]) \bmod q$$

➡️ This avoids recomputing hash from scratch.

Algorithm Steps

1. Compute hash value of **pattern P**
2. Compute hash value of **first window of text**
3. Compare both hash values
4. If equal → compare characters
5. Slide window by one position
6. Update hash using **rolling hash**
7. Repeat till end of text

Example

Text: `ABCCDDAEFG`
Pattern: `CDD`

- Hash of `CDD` is calculated
- Compare with text substrings:
  - `ABC` ❌ `BCC` ❌ `CCD` ❌ `CDD` ☑ **Match found**

Time Complexity

- **Best Case:** `O(n + m)`
- **Average Case:** `O(n + m)`
- **Worst Case:** `O(nm)` (due to hash collisions)

Space Complexity

O(1)

Advantages

- Faster than naïve string matching
- Efficient for **multiple pattern searching**
- Uses **rolling hash**

Disadvantages

- **Hash collisions** possible
- Worst case slower than expected

Applications

- Plagiarism detection

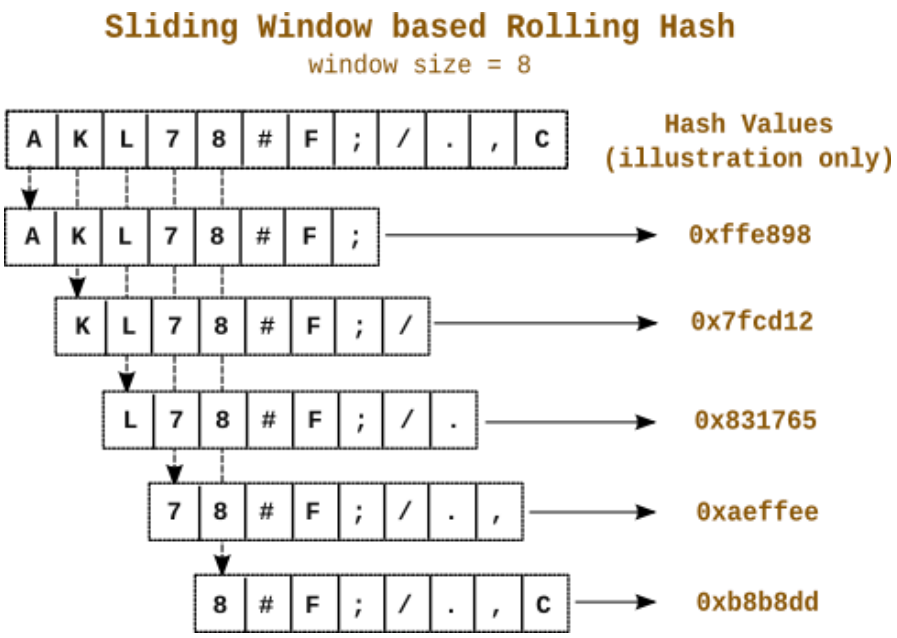
SPPU_TE_BE_COMP 👉 Click to join telegram

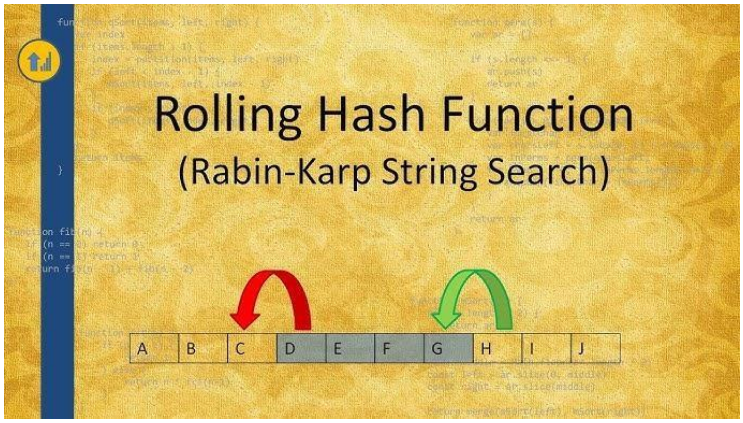
- Virus signature matching
- Text editors (search feature)
- DNA sequence matching

---

15. Rolling hash concept.

Rolling Hash Concept





Definition

Rolling Hash is a hashing technique in which the **hash value of the next substring** is calculated **efficiently from the previous hash value** when the window slides by one position.

👉 It avoids recalculating hash from scratch.

Why Rolling Hash is Needed

- Recomputing hash for every substring is **slow**
- Rolling hash updates hash in **constant time O(1)**
- Used mainly in **Rabin–Karp algorithm**

SPPU_TE_BE_COMP 👈 Click to join telegram

Basic Idea

- Consider a **fixed-size window**
- Slide the window **one character at a time**
- **Remove** the leftmost character
- **Add** the new rightmost character
- Update hash using a formula

Notations

- `T` = Text
- `m` = Length of pattern / window
- `d` = Base (number of characters, usually 10 or 256)
- `q` = Prime number
- `h` = $d^{m-1} \bmod q$

Hash Formula (Initial Window)

$$t = (d \times t + T[i]) \bmod q$$

Rolling Hash Formula (Most Important)

$$t_{next} = (d \times (t - T[i] \times h) + T[i+m]) \bmod q$$

✔ Removes old character
✔ Adds new character
✔ Works in **O(1) time**

Step-by-Step Explanation

1. Calculate hash of first window
2. Slide window by one position
3. Subtract leftmost character's contribution
4. Multiply by base `d`
5. Add new character

6. Apply modulo `q`

Example

Text: `ABCDE`

Window size: `3`

Window	Substring
1	ABC
2	BCD
3	CDE

➡ Hash of `BCD` is calculated from hash of `ABC`

➡ Hash of `CDE` is calculated from hash of `BCD`

No full recomputation ❌

Time Complexity

- Hash update: **O(1)**
- Whole process: **O(n)**

Advantages

- Very fast hash calculation
- Efficient for large texts
- Reduces repeated computation

Disadvantages

- Hash collisions possible
- Needs careful choice of base and prime

SPPU_TE_BE_COMP 👈 Click to join telegram

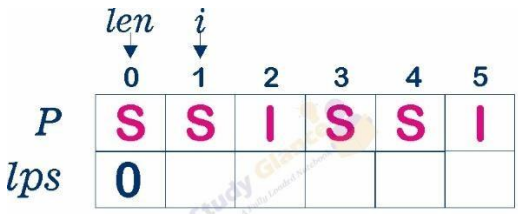
Applications

- Rabin–Karp algorithm
- Plagiarism checking
- String searching
- Data deduplication

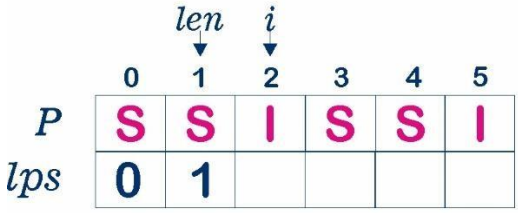
---

16. KMP LPS table.

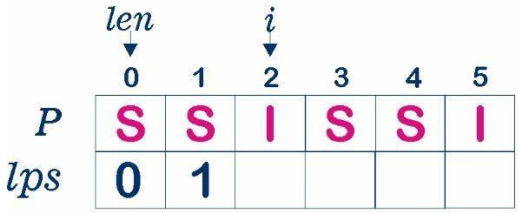
KMP – LPS Table



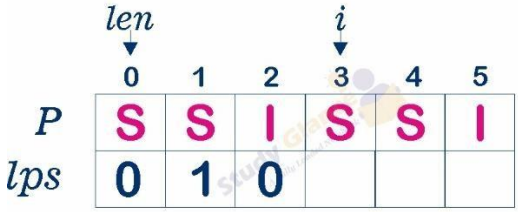
lps[0]=0



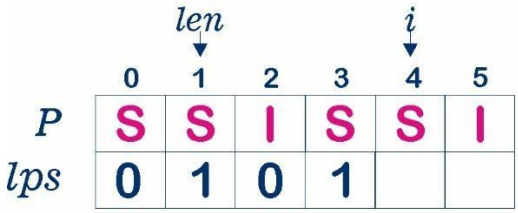
if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



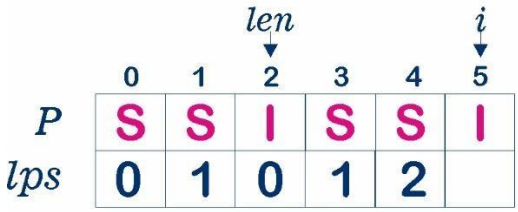
if(p[len] ≠ p[i] and len > 0) then
len = lps[len-1]



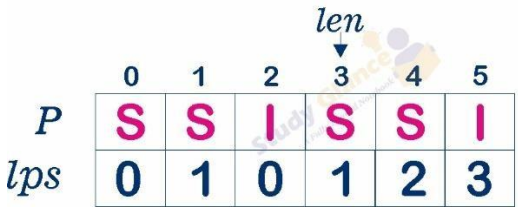
if(p[len] ≠ p[i] and len = 0) then
lps[i] = 0
increment i



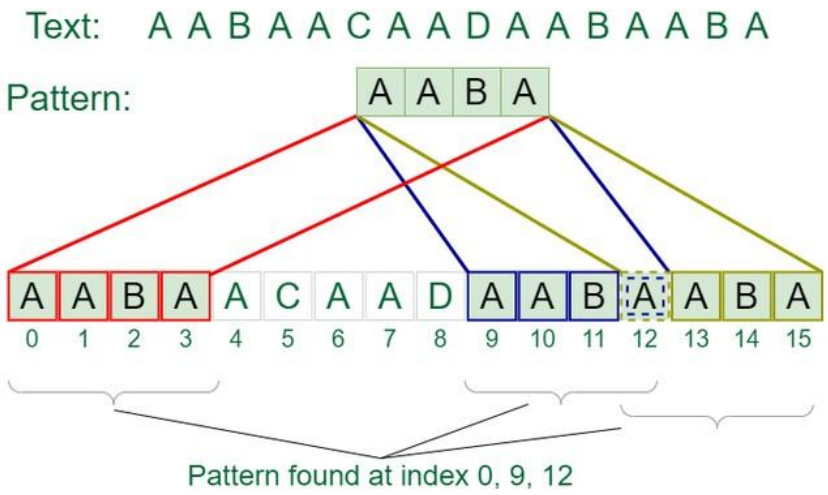
if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



if(p[len] = p[i]) then
lps[i] = len+1
increment len and i



What is LPS?

LPS stands for **Longest Proper Prefix which is also a Suffix**.

- **Proper prefix**: Prefix that is **not equal to the whole string**
- **Suffix**: Ending part of the string

👉 LPS helps **KMP algorithm** avoid unnecessary comparisons.

Definition

LPS[i] stores the **length of the longest proper prefix of the pattern[0…i]** which is also a suffix of this substring.

Why LPS Table is Needed

- Prevents rechecking characters
- Saves time during mismatches
- Makes KMP algorithm efficient

Pattern Example

Pattern: `ABABCABAB`

LPS Table Construction

Index (i)	Character	LPS[i]
0	A	0
1	B	0
2	A	1
3	B	2
4	C	0
5	A	1
6	B	2
7	A	3
8	B	4

How LPS is Calculated (Steps)

1. LPS[0] = 0
2. Compare current character with prefix character
3. If match → increase length and store it
4. If mismatch → fall back using previous LPS value
5. Repeat till end of pattern

Simple Rule to Remember

- **Match → increase length**
- **Mismatch → move back using LPS**
- **Never move pattern index backward in text**

Key Properties

- LPS array size = length of pattern
- LPS values are always **≤ index**

- First value is always **0**

Role of LPS in KMP Algorithm

- On mismatch:
- $$j = LPS[j-1]$$
- Pattern shifts without rechecking matched characters

Time Complexity

- LPS table construction: **O(m)**
- KMP searching: **O(n)**

Advantages

- Avoids redundant comparisons
- Works efficiently for large texts
- Deterministic and fast

---

17. KMP algorithm.

Definition

KMP (Knuth–Morris–Pratt) algorithm is an **efficient string matching algorithm** used to find a **pattern** in a **text** by avoiding unnecessary re-comparisons using the **LPS table**.

Basic Idea

- Preprocess the **pattern** to create **LPS table**

- Use LPS to decide how much to shift the pattern on mismatch
- **Text index never moves backward**

Important Terms

- `T` = Text of length `n`
- `P` = Pattern of length `m`
- `LPS[]` = Longest Prefix Suffix table

Working Principle

1. Compare pattern with text from left to right
2. If characters match → move both pointers 3. If mismatch:
    - If `j ≠ 0` → set `j = LPS[j-1]` ○ If `j = 0` → move text pointer forward
4. Continue until pattern is found or text ends

Example

Text: `ABABDABACDABABCABAB` **Pattern:**

`ABABCABAB`

➡ Pattern found at **index 10**

Formula Used on Mismatch

$$j = LPS[j-1]$$

Algorithm Steps (Exam Style)

1. Construct **LPS table** for the pattern
2. Set `i = 0` (text index), `j = 0` (pattern index)
3. Compare `T[i]` and `P[j]`
4. If match → `i++`, `j++`
5. If `j = m` → pattern found

SPPU_TE_BE_COMP 👉 Click to join telegram

6. If mismatch → update `j` using LPS
7. Repeat till `i < n`

Time Complexity

LPS construction: **O(m)**
Pattern searching: **O(n)**

- **Overall: O(n + m)**

Space Complexity

O(m)

Advantages

- No backtracking of text pointer
- Efficient for large texts
- Better than naïve algorithm

Disadvantages

- LPS table computation is tricky
- More complex than naïve method

Applications

- Text editors (find feature)
- DNA sequence matching
- Pattern searching in files
- Compiler design

---

18. Hashing applications.

-
-

What is Hashing? (1-line recall)

Hashing is a technique that maps data of variable size to a **fixed-size value** using a **hash function** for fast access.

Major Applications of Hashing

---

1. Database Indexing

- Hash tables are used to **quickly locate records**
- Reduces search time from **O(n)** to **O(1)**
- Used in **DBMS indexing**

---

2. Password Storage and Authentication

- Passwords are stored as **hash values**
- Original password cannot be easily retrieved
- Improves **security**

---

3. Symbol Tables (Compilers)

- Used to store **variables, functions, keywords**
- Enables fast **lookup during compilation**

---

4. Caching

- Hashing maps **keys to cache locations** •
  Used in:
  - Web browsers
  - CPU cache ○
    DNS cache

---

5. File Systems

- Hashing helps in **file lookup**
- Used in directory structures
- Faster file searching

---

6. Data Deduplication

Detects **duplicate data**
Same hash → same data
- Used in cloud storage

---

7. String Matching Algorithms

- Used in **Rabin–Karp algorithm**
- Efficient pattern matching using **hash values**

---

8. Cryptography

- Hash functions used in:
  - Digital signatures ○ Data integrity checking
- Example: message authentication

---

9. Load Balancing

- Hashing distributes requests across servers
- Ensures **uniform load distribution**

---

10. Blockchain Technology

- Hashing links blocks securely
- Ensures **immutability** and **security**

Advantages of Hashing

- Very fast access
- Efficient memory usage
- Simple implementation

-
-

Limitations

- Collisions may occur
- Requires good hash function

---

19. String matching applications.

String Matching Applications

String matching is the process of finding occurrences of a **pattern string** within a **larger text string**. It is widely used in computer science and real-world systems.

---

1. Text Searching (Search Engines)

- Used to find keywords in documents, books, PDFs, and web pages.
- Example: Searching a word in Google, Word, or PDF files.
- Algorithms used: **Naïve, KMP, Rabin–Karp**

---

2. Plagiarism Detection

- Compares documents to detect copied content.
- Used in universities and content platforms.
- Matches long patterns across multiple files.

---

3. DNA and Protein Sequence Analysis

- Used in **bioinformatics** to find gene patterns.
- Matches nucleotide sequences like A, T, G, C.
- Helps in disease detection and research.

---

4. Pattern Matching in Compilers

- Used in **lexical analysis**.
- Identifies keywords, identifiers, operators, and symbols.

- Example: Finding `if`, `while`, `int` in source code.

---

5. Spell Checking and Auto-Correction

- Finds closest matching words from dictionary.
  Used in text editors and mobile keyboards.
  Helps suggest correct words for misspelled input.

---

6. Virus and Malware Detection

- Antivirus software scans files for known virus signatures.
- Matches harmful code patterns inside files.

---

7. Data Compression

- Repeated string patterns are detected and encoded efficiently.
- Used in ZIP, RAR, and other compression tools.

---

8. Information Retrieval Systems

- Used in databases to retrieve records based on string queries.
- Example: Searching names, IDs, or keywords in large datasets.

---

9. Chat Applications & Messaging Filters

- Used to detect abusive words or spam messages.
- Filters messages by matching predefined word lists.

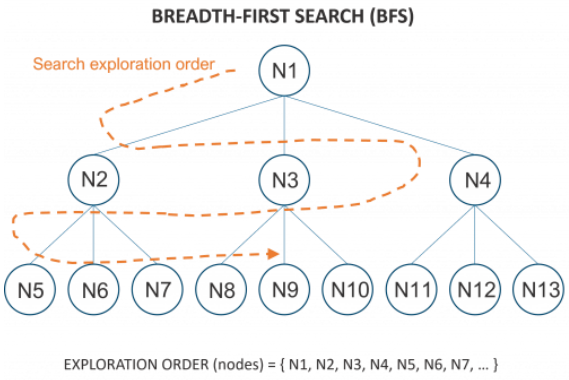
---

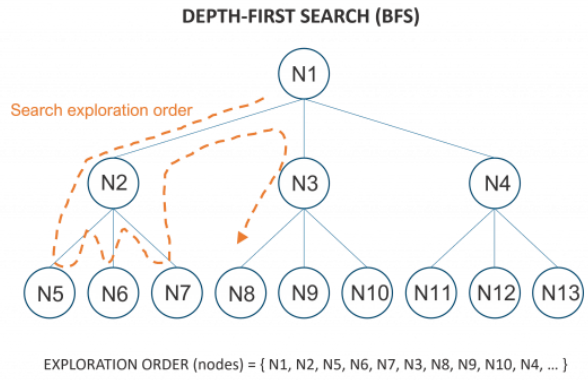
10. Network Security & Intrusion Detection

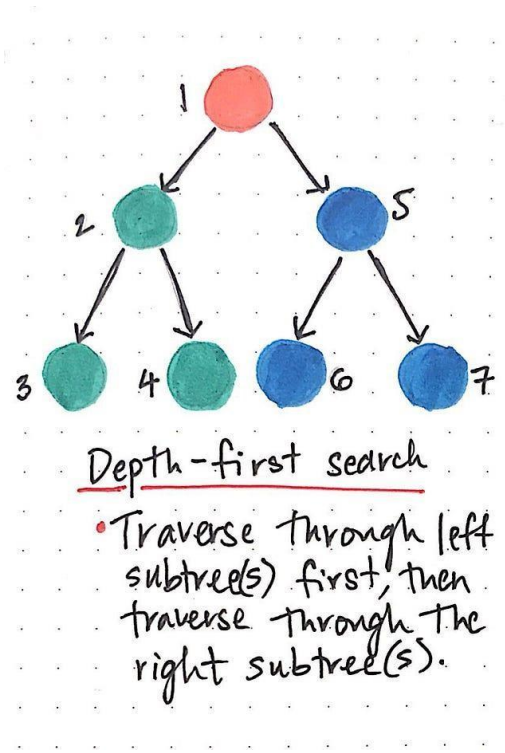
- Detects suspicious patterns in network packets.
- Used in firewalls and IDS systems.

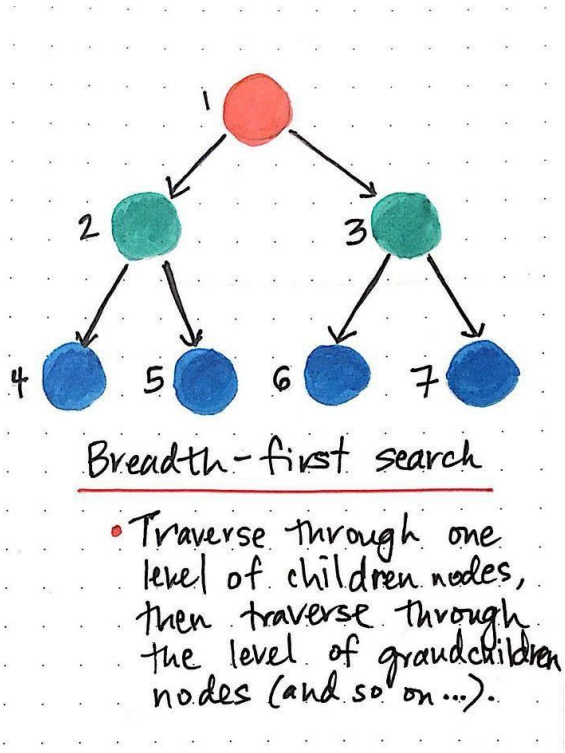
20. DSA for AI search.

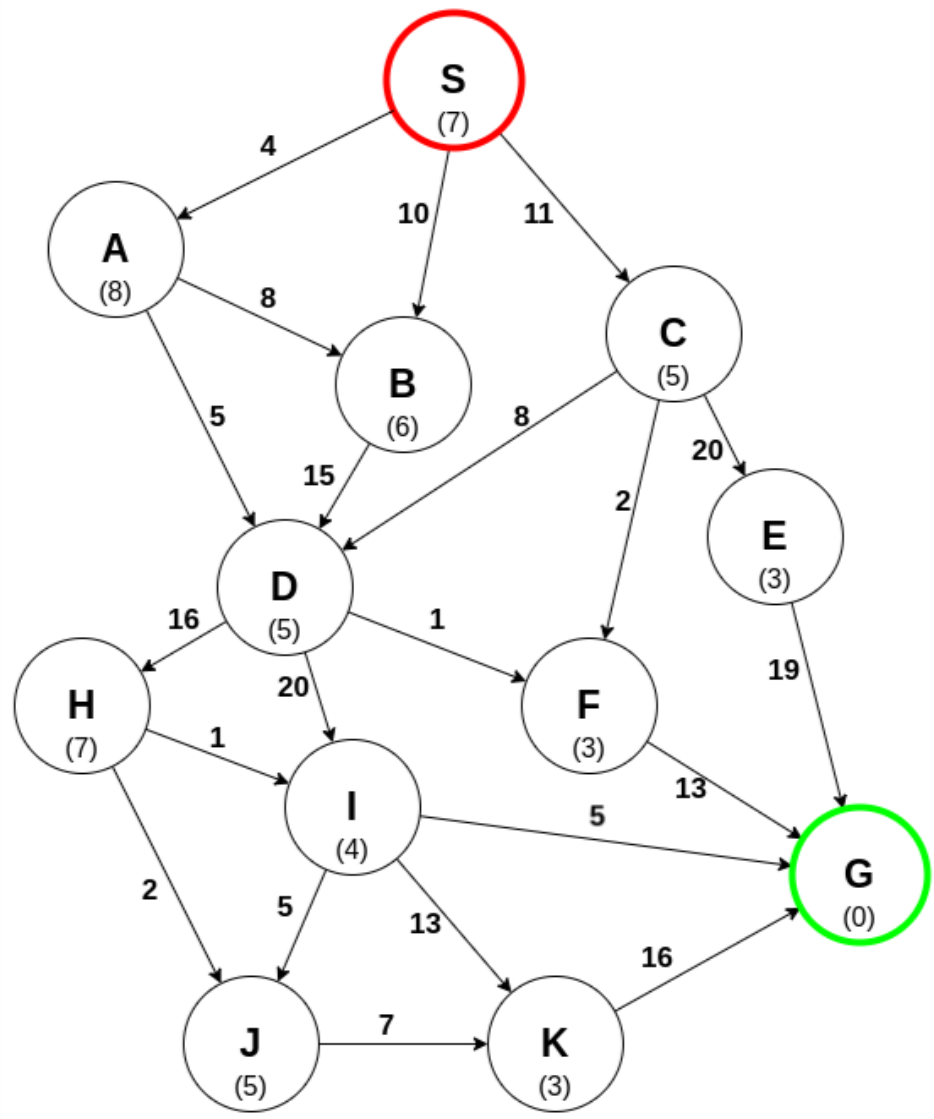
DSA for AI Search











What is AI Search?

In **Artificial Intelligence**, *search* means **finding a sequence of actions** that leads from an **initial state** to a **goal state**.
To perform search efficiently, AI heavily depends on **Data Structures and Algorithms (DSA)**.

Role of DSA in AI Search

DSA helps AI systems to:

- Store states efficiently
- Explore possible solutions systematically
- Reduce time and memory usage
- Find **optimal** or **near-optimal** solutions

Core Data Structures Used in AI Search

1. Stack

- Used in **Depth First Search (DFS)**
- LIFO (Last In First Out)
- Explores one path deeply before backtracking

Application: Puzzle solving, game trees

---

2. Queue

- Used in **Breadth First Search (BFS)**
- FIFO (First In First Out)
- Explores level by level

Application: Shortest path in unweighted graphs

---

3. Priority Queue

- Used in **Best First Search** and **A*** algorithm
- Elements are removed based on **priority (cost or heuristic)**

Application: Pathfinding, robotics, navigation systems

---

4. Graphs

- States are represented as **nodes**
- Actions are represented as **edges**
- Most AI problems are modeled as graphs

Application: Route planning, social networks, knowledge graphs

---

5. Trees

- A special form of graph with hierarchical structure
- Used in **game playing AI**

Application: Minimax algorithm, decision making

6. Hash Tables

- Used to store **visited states**
- Avoids repeated exploration

Application: Optimization and loop prevention

AI Search Algorithms Based on DSA

1. Uninformed (Blind) Search

Uses no extra information about the goal.

Algorithm	Data Structure
BFS	Queue
DFS	Stack
Depth Limited Search	Stack
Iterative Deepening DFS	Stack

---

2. Informed (Heuristic) Search Uses **heuristics** to guide the search.

Algorithm	Data Structure
Best First Search	Priority Queue
A* Search	Priority Queue
Greedy Search	Priority Queue

Example: A* Search (Very Important)

A* uses:

- **g(n)** → cost from start to current node

- **h(n)** → heuristic (estimated cost to goal) **Evaluation function:** `f(n) = g(n) + h(n)` This makes A*:

- Complete
- Optimal (if heuristic is admissible)
- Efficient

Applications of DSA-Based AI Search

1. Pathfinding (Google Maps, GPS)
2. Game AI (Chess, PUBG bots)
3. Robotics navigation
4. Recommendation systems
5. Problem solving (8-puzzle, 15-puzzle)
6. Automated planning systems

NUMERICALS

Insert keys using chaining and linear probing.

Insert Keys Using Chaining and Linear Probing

Assumption (for example)

- **Hash table size = 7**
- **Hash function:**
- $$h(k) = k \bmod 7$$
- **Keys to insert:**

50, 700, 76, 85, 92, 73, 101

1️⃣ Insertion Using Chaining

Concept

- Each hash table index contains a **linked list**
- If a collision occurs, the key is **added to the list** at that index

---

Step-by-Step Insertion

Key	h(k)	Action
50	1	Insert at index 1
700	0	Insert at index 0
76	6	Insert at index 6
85	1	Collision → add to chain at index 1
92	1	Collision → add to chain at index 1
73	3	Insert at index 3
101	3	Collision → add to chain at index 3

Final Hash Table (Chaining)

```
Index 0 → 700
Index 1 → 50 → 85 → 92
Index 2 → —
Index 3 → 73 → 101
Index 4 → —
Index 5 → —
Index 6 → 76
```

Advantages

✔ Simple collision handling

✔ No table overflow ✔ Easy insertion

Disadvantages

✗ Extra memory for linked lists
✗ Slower search if chain is long

2️⃣ Insertion Using Linear Probing

Concept

- Uses **open addressing**
- If a collision occurs, check **next index sequentially** •
  Formula:

$$h_i(k) = (h(k) + i) \bmod m$$

Step-by-Step Insertion

Key	h(k)	Probe Sequence	Inserted At
50	1	1	1
700	0	0	0
76	6	6	6
85	1	1 → 2	2
92	1	1 → 2 → 3	3
73	3	3 → 4	4
101	3	3 → 4 → 5	5

Final Hash Table (Linear Probing)

```
Index : Value
0 : 700
1 : 50
```

```
2 : 85
3 : 92
4 : 73
5 : 101
6 : 76
```

Advantages

✔ No extra memory ✔
Cache-friendly

Disadvantages

✗ Primary clustering
✗ Table can get full
✗ Deletion is complex

Insert keys using quadratic probing (table size 7).

Insert Keys Using Quadratic Probing

Given

- Hash table size (m) = 7
- Hash function:

$$h(k) = k \bmod 7$$

- Quadratic probing formula:

$$h_i(k) = (h(k) + i^2) \bmod 7$$

- Keys to insert:

  50, 700, 76, 85, 92, 73, 101

Step-by-Step Insertion

Key	h(k)	Probing Sequence	Inserted At
50	1	1	1
700	0	0	0
76	6	6	6
85	1	1 → (1+1²)=2	2
92	1	1 → 2 → (1+2²)=5	5
73	3	3	3
101	3	3 → (3+1²)=4	4

Final Hash Table (Quadratic Probing)

```
Index : Value
0 : 700
1 : 50
2 : 85
3 : 73
4 : 101
5 : 92
6 : 76
```

Important Points (Exam Ready)

- Quadratic probing reduces **primary clustering**
- It may still cause **secondary clustering**
- Table size should preferably be **prime** • Probing uses square values: 1^2, 2^2, 3^2, ...

Compute Rabin-Karp hash for "ABCD".

Compute Rabin–Karp Hash for “ABCD”

Assumptions (must be written in exam)

- **Alphabet mapping:**
  A = 1, B = 2, C = 3, D = 4
- **Base (d) = 10**
- **Prime number (q) = 101**
- **Pattern length (m) = 4**

Rabin–Karp Hash Formula

$$H = (d^{m-1} \cdot a_1 + d^{m-2} \cdot a_2 + \cdots + a_m) \bmod q$$

Step-by-Step Calculation

$$H = (10^3 \cdot 1 + 10^2 \cdot 2 + 10^1 \cdot 3 + 10^0 \cdot 4) \bmod 101$$

$$H = (1000 + 200 + 30 + 4) \bmod 101$$

$$H = 1234 \bmod 101$$

$$H = 22$$

Final Answer

$$\boxed{\text{Rabin–Karp hash value of “ABCD”} = 22}$$

Compute KMP LPS table for pattern ABABAC

Compute LPS Table for Pattern ABABAC

Pattern

```
Index : 0  1  2  3  4  5
```

```
Char  : A  B  A  B  A  C
```

What is LPS?

LPS[i] = Length of the **Longest Proper Prefix** which is also a **Suffix** for the substring `pattern[0…i]`.

Step-by-Step LPS Calculation

Index (i)	Substring	LPS Value	Reason
0	A	0	Single character
1	AB	0	No proper prefix = suffix
2	ABA	1	A = A
3	ABAB	2	AB = AB
4	ABABA	3	ABA = ABA
5	ABABAC	0	No prefix matches suffix

Final LPS Table

```
Index : 0  1  2  3  4  5
LPS   : 0  0  1  2  3  0
```

Find all occurrences of “ANA” in “BANANA

Given

- **Text (T)** = `BANANA`
- **Pattern (P)** = `ANA`
- **Length of T** = 6
- **Length of P** = 3

Step-by-Step Matching (Index starts from 0)

Starting Index	Substring of T	Match?
0	BAN	✗
1	ANA	☑
2	NAN	✗
3	ANA	☑

☑ Occurrences Found

- **At index 1**
- **At index 3**

Final Answer

The pattern “ANA” occurs at positions 1 and 3 in the text “BANANA”.